



B.S. Abdur Rahman TM

Crescent

Institute of Science & Technology

Deemed to be University u/s 3 of the UGC Act, 1956

DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING

CSD 2257 - Algorithmic Design Techniques Laboratory

Name :

RRN :

Semester : IV



B.S. Abdur Rahman TM

Crescent

Institute of Science & Technology

Deemed to be University u/s 3 of the UGC Act, 1956

BONAFIDE CERTIFICATE

Certified that this is the bonafide record of the work done
by _____ Register No.
_____ of **IV Semester B. Tech - Artificial
Intelligence & Data science** in the **CSD 2257 - Algorithmic
Design Techniques Laboratory** during the year 2025 - 2026.

Course Faculty

Head of the Department

Date:

Submitted for the Practical Examination held on _____.

Examiner

Program No.: 2

Date:

Linear Search Algorithm

Aim:

To implement the Linear search algorithm.

Algorithm LinearSearch(A[0...n-1], K) :

// Searches for a given key in an array using Linear Search

// Input: An array A[0..n-1] of elements and a key K

// Output: Index of K if found, otherwise -1

BEGIN

FOR i ← 0 TO n-1 DO

IF A[i] = K THEN

RETURN i // Key found at index i

ENDIF

ENDFOR

RETURN -1 // Key not found

END

C Program:

```
#include <stdio.h>
#include <conio.h>
int main()
{
    int arr[20], n, i, key, count = 0, found = 0;
    clrscr();
    printf("Name: Mohammed Omer Akil");
    printf("RRN: 240171601164");
    printf("Enter number of elements: ");
    scanf("%d", &n);
    printf("Enter %d elements:\n", n);
    for (i = 0; i < n; i++) {
        scanf("%d", &arr[i]);
    }
    printf("Enter element to search:");
    scanf("%d", &key);
    for (i=0;i<n;i++) {
        count++;
        if (arr[i]==key) {
            printf("Element %d found at position %d\n", key, i + 1);
            printf("Counter = %d\n", count);
```

```
found = 1;

break;
}
}
if (!found) {
printf("Element not found\n");
printf("Counter = %d\n", count);
}
getch();
return 0;
}
```

Output:

```
Name: Mohammed fardeenRRN: 240171601189Enter number of elements: 5
Enter 5 elements:
11 22 33 2 4
Enter element to search:33
Element 33 found at position 3
Counter = 3

-----
Process exited after 26.86 seconds with return value 0
Press any key to continue . . .
```

Result:

Hence, the implementation of the Linear search algorithm is successfully verified.

Program No.: 3

Date:

Binary Search Algorithm

Aim:

To implement the Binary search algorithm.

Algorithm BinarySearch(A[0...n-1], K) :

// Searches for a given key in a sorted array using Binary Search
// Input: A sorted array A[0...n-1] and a key K
// Output: Index of K if found, otherwise -1

BEGIN

left $\leftarrow$ 0

right $\leftarrow$ n - 1

WHILE left $\leq$ right DO

mid $\leftarrow$ left + (right - left) / 2

IF A[mid] = K THEN

RETURN mid // Key found at index mid

ELSE IF A[mid] < K THEN

left $\leftarrow$ mid + 1 // Search in the right half

ELSE

right $\leftarrow$ mid - 1 // Search in the left half

ENDIF

ENDWHILE

RETURN -1 // Key not found

END

C Program:

```
#include <stdio.h>
```

```
#include <conio.h>
```

```
void main()
```

```
{
```

```
int a[10],i,n,key,count=0;
```

```
int low,high,mid;
```

```
clrscr();
```

```
printf("Name: Mohammed Omer Akil\n");
```

```
printf("RRN: 240171601164\n");
```

```
printf("Enter number of elements:");
scanf("%d", &n);
printf("Enter %d elements in sorted order:\n", n);
for(i=0;i<n;i++)
{
scanf("%d",&a[i]);
}
printf("Enter element to search:");
scanf("%d",&key);
low = 0;
high = n-1;
while(low<=high)
{
mid = (low + high) / 2;
if(a[mid]==key)
{
printf("Element found at position %d",mid+1);
count++;
printf("\nCounter %d",count);
getch();
return;
}
else if(a[mid]<key)
{
low=mid+1;
count++;
}
else
{
high=mid-1;
count++;
}
}
printf("Element not found");
getch();
}
```

Output:

```
Enter number of elements: 5
Enter 5 elements:
23
7
5
77
45

Sorted Array:
5 7 23 45 77

Enter element to search: 23

Element found at position 3

Name: mohammed fardeen
RRN: 240171601189
-----
Process exited after 19.72 seconds with return value 0
Press any key to continue . . . |
```

Result:

Hence, the implementation of the Binary search algorithm is successfully verified.

Program No.: 5

Date:

Selection Sort Algorithm

Aim:

To sort an array of elements using Selection Sort algorithm.

Algorithm SelectionSort(A[0...n-1]):

```
#include <stdio.h>
#include <conio.h>

void selectionSort(int arr[], int n, int *comp) {
    int i, j, k, minIndex, temp;
    for (i = 0; i < n - 1; i++) {
        minIndex = i;
        for (j = i + 1; j < n; j++) {
            (*comp)++;
            if (arr[j] < arr[minIndex]) {
                minIndex = j;
            }
        }
        temp = arr[minIndex];
        arr[minIndex] = arr[i];
        arr[i] = temp;
        printf("Pass %d: ", i + 1);
        for (k = 0; k < n; k++) {
            printf("%d ", arr[k]);
        }
        printf("\n");
    }
}

int main() {
    int n, i;
    int comp = 0;
    int arr[100];
    clrscr();
    printf("Name: Mohammed Omer Akil\n");
    printf("RRN: 240171601164\n");
    printf("Enter the number of elements: ");

    scanf("%d", &n);
    printf("Enter %d elements:\n", n);
    for (i = 0; i < n; i++) {
        scanf("%d", &arr[i]);
```

```
}  
printf("\nSorting process:\n");  
selectionSort(arr, n, &comp);  
printf("\nSorted array: ");  
for (i = 0; i < n; i++) {  
    printf("%d ", arr[i]);  
}  
printf("\nNumber of comparisons: %d", comp);  
getch();  
return 0;  
}
```

Output:

```
Name: Mohammed fardeen  
RRN: 240171601189  
Enter the number of elements: 5  
Enter 5 elements:  
23 56 1 6 28  
  
Sorting process:  
Pass 1: 1 56 23 6 28  
Pass 2: 1 6 23 56 28  
Pass 3: 1 6 23 56 28  
Pass 4: 1 6 23 28 56  
  
Sorted array: 1 6 23 28 56  
Number of comparisons: 10  
-----  
Process exited after 19.55 seconds with return value 0  
Press any key to continue . . . █
```

Result:

Hence, sorting an array of elements using Selection Sort algorithm is successfully verified.

Program No.: 4

Date:

Maximum Element In An Array

Aim:

To find the Maximum Element In an Array

Algorithm Maximum Element In An Array:

START

DECLARE array arr of size 100

DECLARE integers n, i, max

DISPLAY "Enter size of array"

READ n

DISPLAY "Enter n elements"

FOR i $\leftarrow$ 0 TO n - 1

 READ arr[i]

END FOR

SET max $\leftarrow$ arr[0]

FOR i $\leftarrow$ 1 TO n - 1

 IF arr[i] > max THEN

 max $\leftarrow$ arr[i]

 END IF

END FOR

DISPLAY "Maximum value =", max

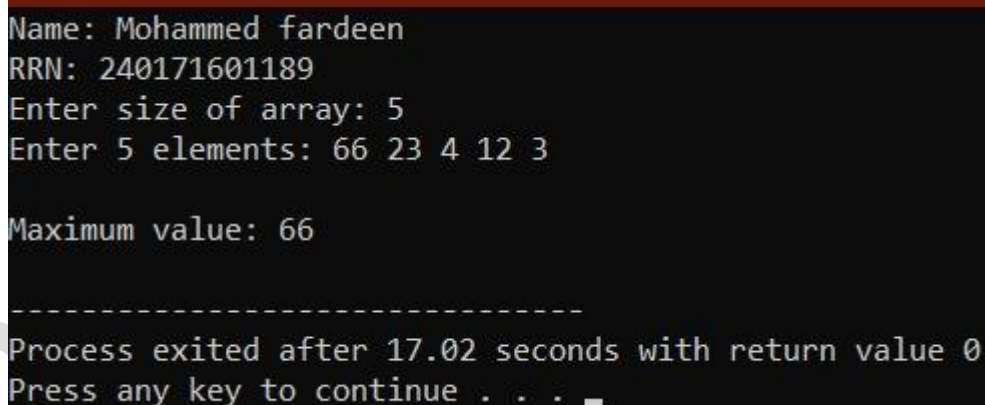
END

C Program:

```
#include <stdio.h>
#include <conio.h>

int main(){
int arr[100], n, i, max;
clrscr();
printf("Name: Mohammed Omer Akil\n");
printf("RRN: 240171601164\n");
printf("Enter size of array: ");
scanf("%d", &n);
printf("Enter %d elements: ", n);
for(i = 0;i<n;i++) {
scanf("%d", &arr[i]);
}
max = arr[0];
for(i=1;i<n;i++) {
if(arr[i]>max) {
max=arr[i];
}
}
printf("\nMaximum value:%d", max);
getch();
return 0;
}
```

Output:

A screenshot of a terminal window showing the execution of a C program. The output is as follows:

```
Name: Mohammed fardeen
RRN: 240171601189
Enter size of array: 5
Enter 5 elements: 66 23 4 12 3

Maximum value: 66

-----
Process exited after 17.02 seconds with return value 0
Press any key to continue . . .
```

Result:

Hence, finding the maximum element in an array is successfully verified.

Program No.: 1A

Date:

Euclidean Algorithm

Aim:

To find the GCD of Two numbers using Euclidean Algorithm.

Algorithm Euclidean Algorithm:

START

DECLARE integers m, n, r

DECLARE count $\leftarrow$ 0

DISPLAY "Enter two integers"

READ m, n

WHILE $n \neq 0$ DO

$r \leftarrow m \text{ MOD } n$

$m \leftarrow n$

$n \leftarrow r$

 count $\leftarrow$ count + 1

END WHILE

DISPLAY "GCD is", m

DISPLAY "Count", count

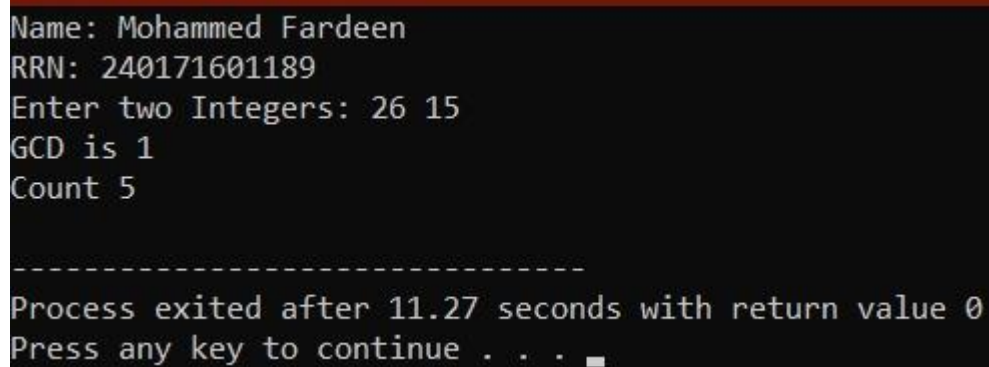
END

C Program:

```
#include<stdio.h>
#include<conio.h>
int main()
{
    int m,n,r;
    int count=0;
    clrscr();
    printf("Name: Mohammed Omer Akil\n");
    printf("RRN: 240171601164\n");
    printf("Enter two Integers:");
    scanf("%d%d",&m,&n);
```

```
while(n!=0){  
r=m%n;  
m=n;  
n=r;  
count++;  
}  
printf("GCD is %d",m);  
printf("Count %d",count);  
getch();  
return 0;  
}
```

Output:



```
Name: Mohammed Fardeen  
RRN: 240171601189  
Enter two Integers: 26 15  
GCD is 1  
Count 5  
  
-----  
Process exited after 11.27 seconds with return value 0  
Press any key to continue . . .
```

Result:

Hence, to find the GCD of two numbers using Euclidean Algorithm is successfully verified.

Program No.: 1B

Date:

GCD Using Prime Factorization

Aim:

To find the GCD of Two numbers using Prime Factorization.

Algorithm:

START

DECLARE integers a, b, i

DECLARE gcd $\leftarrow$ 1

DECLARE count $\leftarrow$ 0

DISPLAY "Enter two numbers"

READ a, b

FOR i $\leftarrow$ 2 TO MIN(a, b) DO

 WHILE (a MOD i = 0) AND (b MOD i = 0) DO

 gcd $\leftarrow$ gcd $\times$ i

 a $\leftarrow$ a / i

 b $\leftarrow$ b / i

 count $\leftarrow$ count + 1

 END WHILE

END FOR

DISPLAY "GCD =", gcd

DISPLAY "Number of comparisons =", count

END

C Program:

```
#include <stdio.h>
```

```
#include <conio.h>
```

```
void main()
```

```
{
```

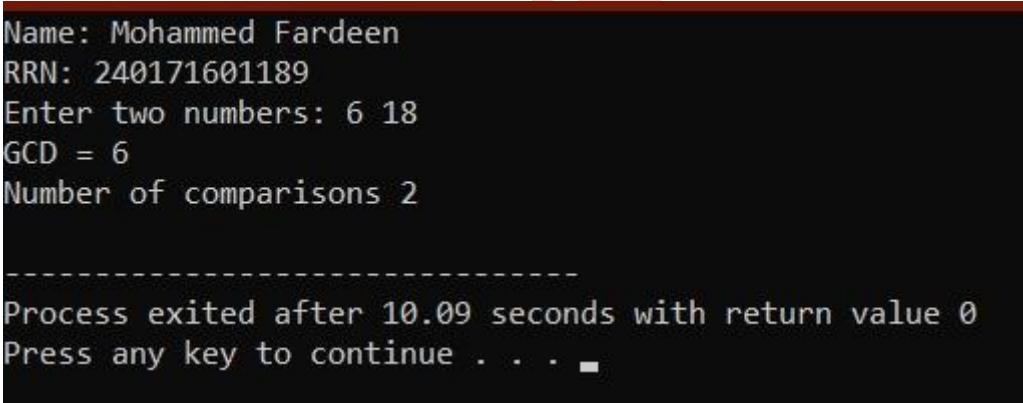
```
int a,b,i,gcd=1,count=0 ;
```

```

clrscr();
printf("Name: Mohammed Omer Akil\n");
printf("RRN: 240171601164\n");
printf("Enter two numbers: ");
scanf("%d %d", &a, &b);
for(i=2;i<=a && i<=b;i++)
{
while(a%i==0 && b%i==0)
{
gcd=gcd*i;
a = a/i;
b = b/i;
count++;
}
}
printf("GCD =%d\n", gcd);
printf("Number of comparisons %d \n",count);
getch();
}

```

Output:



```

Name: Mohammed Fardeen
RRN: 240171601189
Enter two numbers: 6 18
GCD = 6
Number of comparisons 2

-----
Process exited after 10.09 seconds with return value 0
Press any key to continue . . .

```

Result:

Hence, finding the GCD of Two numbers using Prime Factorization is successfully verified.
